

もじれつ 文字列

I. String^{がた}型 (プリミティブ) とStringオブジェクト

プログラミングを学んでいると、同じ「文字列」を扱うのに、**String^{がた}型 (プリミティブ)** とStringオブジェクトという2つの言葉^{ことば}が出てきて混乱^{こんらん}することがあります。

学生^{がくせい}の方^{かた}には、「中身^{なかみ}だけのリンゴ」と「箱^{はこ}に入ったリンゴ」の違^{ちが}いに例^{たと}えて説明^{せつめい}すると、スッキリ理解^{りかい}してもらえるでしょう！

1. 二つの違^{ちが}いのイメージ

- **String^{がた}型 (プリミティブ) :**
- ただの「値^{あたい}」そのものです。
- **例^{たと}え :** お店で売^みっている「リンゴ」そのもの。軽^{かる}くて、すぐ食^たべられます。
- **Stringオブジェクト :**
- `new String()` を使^{つか}って作^{つく}られる「特別な箱^{とくべつ はこ}」です。
- **例^{たと}え :** 「豪華^{ごうか}な箱^{はこ}に入ったリンゴ」です。箱^{はこ}の中^{なか}にはリンゴだけでなく、説明書^{せつめいしょ}や道具^{どうぐ}も一緒^{いっしょ}に入^{はい}っています。

2. コードで見てみよう

実際^{じっさい}にどう違^{ちが}うのか、JavaScriptで書^かいてみます。

```
// 1. String型 (プリミティブ) ★ふだん使うのはこっち！
let str1 = "こんにちは";
console.log(typeof str1); // "string"

// 2. Stringオブジェクト (new を使う)
let str2 = new String("こんにちは");
console.log(typeof str2); // "object"
```

ここが注意^{ちゅうい}！「見た目^みは同じでも中身^{め おな}が違^{ちが}う」

```
let s1 = "hello";
let s2 = new String("hello");
```

```
console.log(s1 == s2); // true (文字の中身は同じ)
console.log(s1 === s2); // false (型が違うのでダメ！)
```

3. 不思議なルール：なぜプリミティブでメソッドが使えるの？

ここで学生からよく「先生、不思議です！」と質問が出ます。

「プリミティブは『ただの値』なのに、なぜ `.length` や `.toUpperCase()` などの『道具（メソッド）』が使えるんですか？」

実は、JavaScriptが裏側で一時的な魔法をかけているんです。これを「ラッパーオブジェクト」と呼びます。

1. あなたが `str1.toUpperCase()` と書く。
2. JavaScriptが「あ、道具を使いたいんだな？」と判断する。
3. 一瞬だけ、プリミティブを「オブジェクトの箱」に入れる。
4. 道具（メソッド）を使い終わったら、すぐに箱を捨てて「プリミティブ」に戻す。

この魔法のおかげで、私たちは重い「オブジェクト」をずっと持っていなくても、必要なときだけ便利な道具を使えるのです。

4. どっちを使えばいい？

学生へのアドバイスは、とてもシンプルです。

- 常に「String型（プリミティブ）」を使いましょう！
- 書き方が簡単（`"` で囲むだけ）。
- メモリを節約できて、動きも速い。
- `new String()` は、特別な理由がない限り、**使わなくて大丈夫**です。

まとめ

特徴	String型 (Primitive)	Stringオブジェクト
作り方	<code>"hello"</code>	<code>new String("hello")</code>
データ型	<code>string</code>	<code>object</code>
スピード	速い	少し遅い（重い）
おすすめ	◎ 毎日使う！	✗ ほとんど使わない

[!TIP]

「ラッパー（Wrapper）」の意味

お菓子やプレゼントを包む「ラッピング（wrapping）」と同じ言葉です。

「値を一時的に包んで守る箱」だと教えると、イメージが定着しやすいですよ！

いかがでしょうか？「普段は身軽なプリミティブ、道具を使うときだけ魔法で変身」という仕組みは伝わりそうでしょうか。

II. 文字列の加工

Stringオブジェクト（文字列）には、文字を自由に操るための便利な「道具（メソッド）」がたくさんあります。

学生の方には、「長い文章という『テープ』を、ハサミで切ったり、シールを貼ったりして加工する」イメージで説明すると伝わりやすいですよ。

よく使われる5つのメソッドを紹介합니다。

1. slice（スライス）：好きなところを切り取る

配列のときと同じ名前ですね！文字列でも「○番目から○番目までを取り出す」ことができます。

```
let text = "JavaScript";
let sub = text.slice(0, 4); // 0番目から4番目の「手前」まで

console.log(sub); // "Java"
```

- **ポイント**：元の `text` は変わりません。新しい「切り抜き」を作るだけです。

2. replace（リプレース）：文字を置き換える

間違った文字を直したり、別の言葉に変えたりします。

```
let greeting = "こんにちは、田中さん";
let newGreeting = greeting.replace("田中", "マリア");

console.log(newGreeting); // "こんにちは、マリアさん"
```

- **覚え方**：「リプレース」は「入れ替える」という意味です。

3. split（スプリット）：バラバラにわけ

長い文字列を、カンマやスペースで区切って「配列（リスト）」に変えます。

```
let tags = "日本,韓国,中国";
let array = tags.split(","); // 「,」のところで切る
```

```
console.log(array); // ["日本", "韓国", "中国"]
```

- 覚え方：「スプリット」は「分裂（ぶんれつ）する」という意味。名簿などを作るときにとっても便利です！

4. toUpperCase / toLowerCase：大文字・小文字にする

英語のアルファベットをすべて大文字、または小文字に変えます。

```
let email = "Maria@example.Com";

console.log(email.toLowerCase()); // "maria@example.com"（すべて小文字）
console.log(email.toUpperCase()); // "MARIA@EXAMPLE.COM"（すべて大文字）
```

- ポイント：ユーザーが入力したメールアドレスをきれいに整理するときによく使います。

5. trim（トリム）：まわりの空白を消す

文字の前後に入ってしまった、いらぬ「スペース」を自動で消してくれます。

```
let input = " マリア ";
let cleanInput = input.trim();

console.log(cleanInput); // "マリア"（きれいに消えた！）
```

- 例え：髪の毛を「トリミング（整える）」するのと同じです。

まとめテーブル（学生向け）

メソッド名	やっていること	たとえ話
slice	一部を取り出す	ハサミで切り取る
replace	文字を入れ替える	修正テープで書き直す
split	配列に分ける	チョコをパキパキ割る
toLowerCase	小文字にする	全部小さく書く
trim	前後の隙間を消す	ムダ毛をカットする

[!TIP]

「元の文字は変わらない」というルール

文字列（String）のメソッドは、すべて「加工したあとの新しい文字」を返します。元の変数を変え

たいときは、`text = text.trim();` のように、もう一度代入する必要があるよ、と教えてあげてください。

いかがでしょうか？「文字列の加工」ができるようになると、ユーザーが入力したデータをきれいに整理できるようになります。

III. 文字の検索

文字列 (String) のなかから「特定の文字」を見つけたり、場所を調べたりするときに欠かせない `charAt` と `indexOf` について説明します。

どちらも「何番目にあるか」という **インデックス (番号)** をキーワードにして動きます。

1. charAt (キャラット) : 番号から「文字」を知る

指定した番号の場所に、どんな文字があるかを教えてください。

- **イメージ** : 「3番の部屋に住んでいるのは誰 (どの文字) ?」と聞く感じです。
- **書きかた** : `文字列.charAt(インデックス)`

```
let word = "JavaScript";

console.log(word.charAt(0)); // "J" (一番最初)
console.log(word.charAt(4)); // "S" (5番目)
```

[!TIP]

最近のJavaScriptでは、配列と同じように `word[0]` と書いても同じ結果が得られますが、`charAt` は古いコードでもよく使われる大切な基本です。

2. indexOf (インデックスオブ) : 文字から「番号」を知る

探したい文字が、最初から数えて**何番目 (インデックス) にあるか**を教えてください。

- **イメージ** : 「『S』さんは何番の部屋に住んでいますか？」と聞く感じです。
- **書きかた** : `文字列.indexOf("探したい文字")`

```
let text = "Hello World";

console.log(text.indexOf("W")); // 6 (Wは7番目、つまりインデックスは6)
console.log(text.indexOf("o")); // 4 (最初に見つかった「o」の場所)
```

見つからなかったらどうなる？

ここがテストや実務（じつむ）で一番大切なポイントです。探している文字がどこにもなかった場合、`indexOf` は `-1` という数字を返します。

```
let msg = "こんにちは";
console.log(msg.indexOf("さようなら")); // -1
```

3. 二つのメソッドの違い（まとめ）

学生の方には、この「逆（ぎゃく）」の関係を伝えてあげてください。

メソッド名	知りたいこと（答え）	入力するもの	例え話
<code>charAt</code>	文字 はなに？	番号 (Index)	「0番の席に座っているのは誰？」
<code>indexOf</code>	番号 はどこ？	文字 (String)	「マリアさんは何番の席？」

学生へのアドバイス：`indexOf` の便利な使い方

「あるかないか」を調べるために、`if` 文でよくこう書きます。

```
let email = "test@example.com";

// 「@」が入っているかチェックする
if (email.indexOf("@") !== -1) {
  console.log("正しいメールアドレスの形式（けいしき）です。");
} else {
  console.log("「@」がありません！");
}
```

[!NOTE]

「-1 じゃない (`!== -1`)」 = 「どこかにある」という意味だよ！と教えると、コードの意味がスッと伝わります。

いかがでしょうか？「番号から文字」か「文字から番号」か、このペアで覚えるととても使いやすくなります。

IV. 文字列の応用

Webサイトで扱う情報の多くは「テキスト（文字列）」です。これをどう操作するかが、プログラミングの腕の見せ所です。

文字列の作成と結合

最新のJavaScriptでは、テンプレートリテラル（「`」 バッククォート）を使うのがとても便利です。

- 変数の埋め込み: `こんにちは、\${name}さん!` のように、`\${}` で変数を入れられます。
- 改行（かいぎょう）: テンプレートリテラルは、コードの中で改行した場所をそのまま文字として扱ってくれます。HTMLのコードを文字として作りたいときに非常に役立ちます。

数値への変換

prompt()（プロンプト：入力画面）でユーザーが入力した数字は、見た目が数字でも、プログラム上は「文字列」です。計算するには変換が必要です。

- parseInt(): 整数（1, 2, 3...）に変換します。
- parseFloat(): 小数点を含む数値に変換します。

So What?（なぜ大切なの？） 数字ではない文字を無理に数値に変えようすると NaN（Not a Number：数値ではないというエラー値）になります。Web開発ではユーザーが何を打つか分かりません。isNaN() 関数を使って「これは本当に数字かな？」とチェックすることは、プログラムが止まってしまう「クラッシュ」を防ぐためのプログラマーの責任です。